Laboratório Bancos de Dados Chave-Valor:

Redis vs MySQL

1. Contextualização

Aplicações modernas precisam armazenar e recuperar grandes volumes de dados com rapidez. Entretanto, nem todos os problemas de armazenamento precisam ser resolvidos utilizando exclusivamente bancos de dados relacionais.

Bancos de dados **chave-valor**, como o Redis, utilizam uma estrutura simples na qual uma chave identifica diretamente um valor armazenado. Essa característica pode proporcionar operações muito rápidas quando a aplicação conhece previamente a chave que deseja consultar.

O objetivo é construir uma pequena aplicação de dados utilizando **Python e Redis**, modelar as corridas como estruturas chave-valor e, posteriormente, implementar a mesma consulta utilizando **MySQL**.

Ao final, realizaremos um experimento comparativo para responder à seguinte questão:

Em quais situações o uso de um banco de dados chave-valor como o Redis pode proporcionar vantagens de desempenho em relação a um banco de dados relacional como o MySQL?

2. Dataset — TaxiGov

Nesta atividade, será utilizado o conjunto de dados público **TaxiGov**, que contém informações relacionadas a corridas de táxi. O dataset oficial pode ser encontrado no repositório de dados do Governo Federal:

Repositório oficial do TaxiGov:

Dataset TaxiGov — Repositório de Dados do Governo Federal

https://repositorio.dados.gov.br/seges/taxigov/v2/taxigov-corridas-completo.zip

No repositório estão disponíveis arquivos como:

- `taxigov-corridas-2017.zip`
- `taxigov-corridas-2018.zip`
- `taxigov-corridas-2019.zip`
- `taxigov-corridas-2020.zip`
- `taxigov-corridas-2021.zip`
- `taxigov-corridas-2022.zip`
- `taxigov-corridas-2023.zip`
- `taxigov-corridas-2024.zip`
- `taxigov-corridas-7-dias.zip`
- `taxigov-corridas-completo.zip`

O arquivo consolidado é recomendado para a etapa de benchmark, enquanto um arquivo anual pode ser utilizado inicialmente para testar o código.

Também está disponível no repositório o **dicionário de dados**, que deverá ser consultado antes da construção do modelo.

3. Objetivos

Ao final da atividade, desenvolveremos as seguintes habilidades:

1. Compreender o conceito de banco de dados chave-valor.
2. Utilizar os principais comandos do Redis.
3. Manipular Redis utilizando Python.
4. Trabalhar com um dataset público de grande volume.
5. Modelar registros do TaxiGov em estruturas Redis.
6. Utilizar Hashes, Sets e Sorted Sets.
7. Utilizar pipelines para realizar cargas em lote.
8. Criar uma representação equivalente dos dados em MySQL.
9. Desenvolver um experimento de benchmark.
10. Comparar Redis e MySQL sob uma carga de consultas.
11. Identificar vantagens e limitações dos bancos chave-valor.
12. Avaliar situações em que uma arquitetura híbrida Redis + MySQL pode ser adequada.

4. Tecnologias

Utilize:

- Python 3
- Redis
- MySQL
- Redis CLI ou Redis Insight
- pandas
- redis-py
- mysql-connector-python
- matplotlib

A biblioteca `redis-py` é o cliente Python recomendado na documentação do Redis e oferece acesso aos comandos e estruturas de dados do Redis.

Para MySQL, será utilizado o **MySQL Connector/Python**, biblioteca oficial para comunicação entre aplicações Python e o servidor MySQL.

Instalação:

```
pip install pandas redis mysql-connector-python matplotlib
```

5. Etapa 1 — Conhecendo o Redis

Antes de utilizar Python, precisamos conhecer os comandos diretamente no Redis.

Execute:

```
redis-cli
```

5.1 Strings

Crie uma informação simples:

```
SET taxi:teste "corrida de taxi"
```

Consulte:

```
GET taxi:teste
```

Verifique se existe:

```
EXISTS taxi:teste
```

Apague:

```
DEL taxi:teste
```

6. Etapa 2 — Trabalhando com Hashes

Um Hash do Redis permite representar um objeto composto por campos e valores.

Esse recurso é particularmente interessante para representar uma corrida de táxi. A documentação do Redis apresenta Hashes justamente como uma estrutura de pares campo-valor adequada para representar objetos.

Exemplo:

```
HSET taxi:corrida:1 \
     passageiro "001" \
     origem "Rio de Janeiro" \
     destino "Niterói" \
     valor "45.50"
```

Consulte um campo:

```
HGET taxi:corrida:1 origem
```

Consulte todos:

```
HGETALL taxi:corrida:1
```

Verifique a existência:

```
EXISTS taxi:corrida:1
```

Apague:

```
DEL taxi:corrida:1
```

Exercício

Crie manualmente três corridas:

```
taxi:corrida:1
taxi:corrida:2
taxi:corrida:3
```

Cada uma deverá possuir pelo menos:

- identificador;
- data;
- origem;
- destino;
- valor;

- distância.

7. Etapa 3 — Sets

Agora será criado um conjunto de bairros.

Exemplo:

```
SADD taxi:bairros "Copacabana"
SADD taxi:bairros "Centro"
SADD taxi:bairros "Botafogo"
SADD taxi:bairros "Tijuca"
```

Consulte:

```
SMEMBERS taxi:bairros
```

Quantidade:

```
SCARD taxi:bairros
```

Verifique um bairro:

```
SISMEMBER taxi:bairros "Copacabana"
```

Pergunta
Por que um Set pode ser mais apropriado que uma lista comum para representar um conjunto de bairros únicos?

8. Etapa 4 — Sorted Sets

Imagine que queremos manter um ranking dos bairros de acordo com a quantidade de corridas.

Podemos utilizar um Sorted Set:

```
ZADD taxi:ranking 1500 "Copacabana"
ZADD taxi:ranking 1200 "Centro"
ZADD taxi:ranking 950 "Botafogo"
ZADD taxi:ranking 800 "Tijuca"
```

Consulte:

```
ZRANGE taxi:ranking 0 -1 WITHSCORES
```

Ordem decrescente:

```
ZREVRANGE taxi:ranking 0 -1 WITHSCORES
```

9. Etapa 5 — Contadores

O Redis também permite utilizar operações de incremento.

Exemplo:

```
SET taxi:contador:copacabana 0
```

Depois:

```
INCR taxi:contador:copacabana
```

Execute várias vezes e consulte:

```
GET taxi:contador:copacabana
```

Discussão

Esse recurso pode ser interessante em aplicações que precisam manter contadores de:

- número de corridas;
- número de usuários;
- quantidade de acessos;
- quantidade de solicitações;
- quantidade de eventos.

10. Etapa 6 — Expiração de dados

Crie uma informação temporária:

```
SET taxi:cache:teste "dados temporarios"
```

Configure sua expiração:

```
EXPIRE taxi:cache:teste 60
```

Verifique o tempo restante:

```
TTL taxi:cache:teste
```

O Redis poderá ser utilizado, por exemplo, para armazenar informações temporárias ou caches que não precisam permanecer indefinidamente.

11. Etapa 7 — Conhecendo o TaxiGov

Baixe inicialmente um arquivo menor, por exemplo:

```
taxigov-corridas-2024.zip
```

Depois, para o benchmark final, utilize:

```
taxigov-corridas-completo.zip
```

O repositório oficial fornece também o dicionário de dados. Antes de programar, consulte esse documento para compreender o significado das colunas.

Após descompactar o arquivo, identifique:

1. formato do arquivo;
2. quantidade de registros;
3. quantidade de atributos;
4. identificador da corrida;
5. campos relacionados a data/hora;
6. informações de origem/destino;
7. informações de valor;
8. informações de distância;
9. possíveis campos nulos.

12. Etapa 8 — Explorando o dataset com Python

Utilize o pandas:

```
import pandas as pd

df = pd.read_csv("taxigov.csv")

print(df.head())
print(df.columns)
print(df.shape)
print(df.info())
```

Verifique a quantidade de registros:

```
print("Quantidade de registros:", len(df))
```

Verifique valores ausentes:

```
print(df.isnull().sum())
```

Atividade: Produza um pequeno relatório:

```
Quantidade de registros:
Quantidade de atributos:
Tipos de dados:
Quantidade de valores nulos:
Quantidade de identificadores únicos:
```

13. Etapa 9 — Definindo o modelo Redis

Cada corrida será representada por uma chave.

Adote o seguinte padrão:

```
taxi:corrida:<id>
```

Por exemplo:

```
taxi:corrida:10001
taxi:corrida:10002
taxi:corrida:10003
```

Cada chave deverá conter um Hash.

Exemplo:
```
taxi:corrida:10001

id = 10001
data = 2024-01-15
origem = ...
destino = ...
valor = 35.50
distancia = 8.2
```

Essa estrutura representa diretamente a ideia de banco chave-valor:

```
CHAVE → VALOR
```

onde o valor, nesse caso, é um Hash contendo os atributos da corrida.

14. Etapa 10 — Conectando Python ao Redis

Crie:

```
import redis

r = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True
)

print(r.ping())
```

Se estiver funcionando, deverá aparecer:

```
True
```

15. Etapa 11 — Inserindo uma corrida

Suponha que o identificador seja `10001`.

```
dados = {
    "id": "10001",
    "origem": "Centro",
    "destino": "Copacabana",
    "valor": "35.50",
    "distancia": "8.2"
}

r.hset(
    "taxi:corrida:10001",
    mapping=dados
)
```

Consulta:

```
print(
    r.hgetall("taxi:corrida:10001")
)
```

16. Etapa 12 — Carregando o TaxiGov no Redis

Crie um programa chamado:

```
carregar_redis.py
```

Estrutura inicial:

```
import pandas as pd
import redis

df = pd.read_csv("taxigov.csv")

r = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True
)

for _, row in df.iterrows():

    corrida_id = row["id"]

    chave = f"taxi:corrida:{corrida_id}"
```

```
    dados = {
        "id": str(corrida_id),
        "origem": str(row["origem"]),
        "destino": str(row["destino"]),
        "valor": str(row["valor"]),
        "distancia": str(row["distancia"])
    }

    r.hset(chave, mapping=dados)

print("Carga concluída.")
```

Atenção: os nomes das colunas devem ser adaptados ao arquivo efetivamente baixado e ao dicionário de dados do TaxiGov.

17. Etapa 13 — Melhorando a carga com Pipeline

A carga anterior envia comandos individualmente. Iremos utilizar um **pipeline**.

Exemplo:

```
pipe = r.pipeline(transaction=False)

contador = 0

for _, row in df.iterrows():

    corrida_id = row["id"]

    chave = f"taxi:corrida:{corrida_id}"

    dados = {
        "id": str(corrida_id),
        "origem": str(row["origem"]),
        "destino": str(row["destino"]),
        "valor": str(row["valor"]),
        "distancia": str(row["distancia"])
    }

    pipe.hset(chave, mapping=dados)

    contador += 1

    if contador % 1000 == 0:
        pipe.execute()

pipe.execute()
```

Pipelines permitem agrupar diversos comandos e reduzir o custo de múltiplas comunicações entre aplicação e servidor.

Experimento

Compare:

Carga A	**Carga B**
HSET HSET HSET HSET	PIPELINE HSET HSET HSET

...	... EXECUTE

Registre o tempo das duas operações.

18. Etapa 14 — Criando índices adicionais no Redis

Agora vamos explorar uma característica importante do Redis. Suponha que desejamos descobrir todas as corridas associadas a determinado bairro. Podemos criar um Set:

```
taxi:bairro:copacabana
```

e adicionar os identificadores:

```
SADD taxi:bairro:copacabana 10001
SADD taxi:bairro:copacabana 10025
SADD taxi:bairro:copacabana 10087
```

Depois:

```
SMEMBERS taxi:bairro:copacabana
```

A atividade deverá criar pelo menos um índice adicional desse tipo.

19. Etapa 15 — Criando um ranking

Crie um ranking de bairros.

A cada corrida processada, incremente o número de ocorrências:

```
r.zincrby(
    "taxi:ranking:bairros",
    1,
    bairro
)
```

Depois consulte:

```
ranking = r.zrevrange(
    "taxi:ranking:bairros",
    0,
    9,
    withscores=True
)

print(ranking)
```

O resultado deverá apresentar os bairros com maior quantidade de ocorrências.

20. Etapa 16 — Criando a base equivalente no MySQL

Agora devemos construir uma representação relacional equivalente.

Crie:

```
CREATE DATABASE taxigov;

USE taxigov;
```

Tabela simplificada:

```
CREATE TABLE corrida (
    id BIGINT PRIMARY KEY,
    origem VARCHAR(255),
    destino VARCHAR(255),
    valor DECIMAL(10,2),
    distancia DECIMAL(10,2)
);
```

Dependendo dos campos existentes no arquivo, a tabela poderá ser ampliada.

21. Etapa 17 — Conectando Python ao MySQL

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="SUA_SENHA",
    database="taxigov"
)

cursor = conn.cursor()

print("Conexão realizada.")
```

22. Etapa 18 — Inserindo dados no MySQL

Utilize `executemany()` para inserir vários registros:

```
sql = """
INSERT INTO corrida
(id, origem, destino, valor, distancia)
VALUES (%s, %s, %s, %s, %s)
"""

dados = [
    (
        10001,
        "Centro",
        "Copacabana",
        35.50,
        8.2
    ),
    (
        10002,
        "Botafogo",
        "Tijuca",
        42.30,
        10.5
    )
]

cursor.executemany(sql, dados)

conn.commit()
```

O Connector/Python oferece `executemany()` para executar uma mesma operação sobre várias sequências de parâmetros e possui otimização para inserções em múltiplas linhas.

23. Etapa 19 — Criando um índice no MySQL

Para tornar a comparação mais adequada, garanta que o identificador da corrida seja uma chave primária:

```
ALTER TABLE corrida
ADD PRIMARY KEY (id);
```

O objetivo é comparar duas estruturas que permitam buscar diretamente uma corrida pelo seu identificador:

Redis

```
taxi:corrida:10001
```

MySQL

```
SELECT *
FROM corrida
WHERE id = 10001;
```

24. Etapa 20 — Primeira comparação

Agora será realizada a primeira comparação.

Redis

```
import time

inicio = time.perf_counter()

resultado = r.hgetall(
    "taxi:corrida:10001"
)

fim = time.perf_counter()

print("Redis:", fim - inicio)
print(resultado)
```

MySQL

```
import time

inicio = time.perf_counter()

cursor.execute(
    """
    SELECT id, origem, destino, valor,
distancia
    FROM corrida
    WHERE id = %s
    """,
    (10001,)
)

resultado = cursor.fetchone()

fim = time.perf_counter()

print("MySQL:", fim - inicio)
print(resultado)
```

25. Etapa 21 — Benchmark com 10.000 consultas

Agora será realizado um experimento mais representativo. Selecione aleatoriamente **10.000 identificadores** existentes.

```
import random

ids = df["id"].dropna().astype(int).tolist()

ids_teste = random.choices(
    ids,
    k=10000
)
```

Faça 10.000 consultas no Redis:

```
inicio = time.perf_counter()
```

```
for id_ in ids_teste:

    chave = f"taxi:corrida:{id_}"

    r.hgetall(chave)

fim = time.perf_counter()

tempo_redis = fim - inicio

print("Tempo Redis:", tempo_redis)
```

Faça o mesmo no MySQL:

```
inicio = time.perf_counter()

for id_ in ids_teste:

    cursor.execute(
        """
        SELECT id, origem, destino, valor, distancia
        FROM corrida
        WHERE id = %s
        """,
        (id_,)
    )

    cursor.fetchone()

fim = time.perf_counter()

tempo_mysql = fim - inicio

print("Tempo MySQL:", tempo_mysql)
```

26. Etapa 22 — Calculando consultas por segundo

Calcule:

```
qps_redis = 10000 / tempo_redis
qps_mysql = 10000 / tempo_mysql

print("Redis:", qps_redis, "consultas/s")
print("MySQL:", qps_mysql, "consultas/s")
```

A métrica permitirá responder:

Quantas consultas por segundo cada tecnologia conseguiu executar nesse experimento?

27. Etapa 23 — Calculando o ganho de desempenho

Utilize:

```
ganho = (
    (tempo_mysql - tempo_redis)
    / tempo_mysql
) * 100

print(
    f"Ganho do Redis: {ganho:.2f}%"
)
```

Por exemplo, se:

```
MySQL = 5 segundos
Redis = 2 segundos
```

então:

```
Ganho = 60%
```

Isso significa que, **nesse experimento específico**, o Redis levou 60% menos tempo que o MySQL.

28. Etapa 24 — Variando o tamanho do experimento

Repita o benchmark com:

Número de consultas
1.000
10.000
50.000
100.000
500.000

Para cada experimento, registre:

Consultas	Redis (s)	MySQL (s)	Redis QPS	MySQL QPS
1.000				
10.000				
50.000				
100.000				
500.000				

29. Etapa 25 — Construindo um gráfico

Utilize Python:

```
import matplotlib.pyplot as plt

consultas = [
    1000,
    10000,
    50000,
    100000,
    500000
]

redis_tempo = [
    # preencher
]

mysql_tempo = [
    # preencher
]

plt.plot(
    consultas,
    redis_tempo,
    marker="o",
```

```
    label="Redis"
)

plt.plot(
    consultas,
    mysql_tempo,
    marker="o",
    label="MySQL"
)

plt.xlabel("Número de consultas")
plt.ylabel("Tempo (segundos)")
plt.title("Redis x MySQL — Tempo de consulta")

plt.legend()
plt.grid()

plt.show()
```

30. Etapa 26 — Redis individual x Redis Pipeline

Faça uma segunda comparação.

Estratégia A

Uma consulta por vez:

```
for id_ in ids_teste:
    r.hgetall(
        f"taxi:corrida:{id_}"
    )
```

Estratégia B

Utilizando Pipeline:

```
pipe = r.pipeline(
    transaction=False
)

for id_ in ids_teste:

    pipe.hgetall(
        f"taxi:corrida:{id_}"
    )

resultados = pipe.execute()
```

Compare os tempos.

O objetivo é demonstrar que parte do desempenho percebido pode estar relacionada não somente ao banco utilizado, mas também à forma como a aplicação conversa com o banco. A documentação do Redis destaca justamente a redução de múltiplas comunicações proporcionada pelo pipelining.

31. Etapa 27 — Testando diferentes padrões de acesso

Agora responda experimentalmente:

Experimento A

Buscar uma corrida pelo ID.

`taxi:corrida:10001`

Experimento B

Buscar 10.000 corridas pelo ID.

Experimento C

Buscar todas as corridas de determinado bairro.

Experimento D

Obter os 10 bairros com maior número de corridas.

Experimento E

Calcular o valor médio das corridas.

32. Etapa 28 — Comparando as capacidades

Monte uma tabela:

Operação	Redis	MySQL
Buscar corrida pelo ID		
Buscar várias corridas pelo ID		
Agrupamento por bairro		
Média de valores		
JOIN entre tabelas		
Ranking pré-computado		
Contador		
Cache		
Dados temporários com TTL		
Consultas SQL complexas		

Preencha a tabela e justifique suas respostas.

33. Questão central da atividade

Após os experimentos, responda:

Se o Redis apresentou desempenho superior ao MySQL no benchmark, significa que o Redis é um banco de dados melhor que o MySQL? Justifique.

A resposta deverá considerar:

- modelo de dados;
- tipo de consulta;
- quantidade de dados;
- memória disponível;
- persistência;
- complexidade das consultas;
- necessidade de relacionamentos;

- necessidade de agregações;
- necessidade de transações;
- padrão de acesso da aplicação.

34. Discussão sobre as limitações do Redis

Hora de identificar pelo menos **cinco limitações ou desafios** do uso do Redis nesse cenário.

Alguns pontos que podem ser investigados:

34.1 Memória

O Redis trabalha fortemente orientado a estruturas em memória, portanto grandes volumes podem representar um custo significativo de infraestrutura.

34.2 Consultas complexas

Uma consulta como:

```
SELECT bairro, AVG(valor)
FROM corrida
GROUP BY bairro;
```

é naturalmente expressa em SQL.

No Redis, pode ser necessário criar estruturas auxiliares, realizar processamento na aplicação ou manter agregações pré-computadas.

34.3 Relacionamentos

MySQL oferece mecanismos naturais para trabalhar com relacionamentos entre tabelas. No modelo chave-valor, o desenvolvedor precisa modelar explicitamente essas relações.

34.4 Redundância

Para tornar determinadas consultas rápidas, pode ser necessário manter índices ou estruturas adicionais no Redis. Isso pode aumentar a quantidade de dados armazenados.

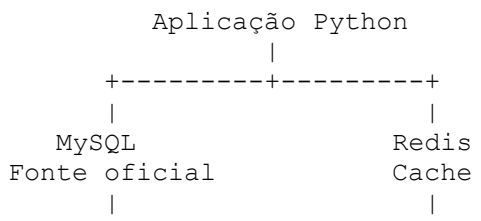
34.5 Consistência

Quando o Redis é utilizado como cópia/cache de dados existentes em outro banco, surge o problema de manter as informações sincronizadas.

35. Atividade adicional — Arquitetura híbrida

Imagine que uma empresa precise desenvolver um sistema de gestão de transporte.

Proponha uma arquitetura utilizando:



```
Dados persistentes    Consultas rápidas
```

1. Quais dados permaneceriam no MySQL?
2. Quais dados poderiam ser armazenados no Redis?
3. O que poderia ser colocado em cache?
4. Quais dados poderiam utilizar TTL?
5. Que informações poderiam ser armazenadas em Sorted Sets?
6. Como atualizar o Redis quando o MySQL fosse alterado?
7. O que aconteceria se o Redis fosse reiniciado?
8. Qual banco seria considerado a fonte oficial dos dados?

36. Desafio final

Implemente um pequeno sistema de consulta de corridas.

O sistema deverá receber:

```
ID da corrida
```

e retornar:

```
ID
Origem
Destino
Valor
Distância
```

Implemente duas versões:

Versão 1 — Redis

```
Python → Redis → Hash
```

Versão 2 — MySQL

```
Python → MySQL → SELECT
```

O programa deverá permitir escolher:

```
1 - Consultar no Redis
2 - Consultar no MySQL
3 - Executar benchmark
4 - Mostrar ranking de bairros
5 - Sair
```

37. Entregáveis

Cada grupo deverá entregar:

1. Código Python

Incluindo:

- leitura do TaxiGov;
- tratamento dos dados;
- carga no Redis;

- carga no MySQL;
- consultas Redis;
- consultas MySQL;
- benchmark;
- geração dos gráficos.

2. Bancos de dados

Redis e MySQL deverão estar populados com a amostra utilizada.

3. Relatório

O relatório deverá conter:

1. descrição do dataset;
2. quantidade de registros utilizada;
3. modelo Redis;
4. modelo MySQL;
5. comandos Redis utilizados;
6. processo de carga;
7. comparação entre carga individual e pipeline;
8. metodologia do benchmark;
9. resultados;
10. gráficos;
11. análise dos resultados;
12. vantagens do Redis;
13. limitações do Redis;
14. vantagens do MySQL;
15. proposta de arquitetura híbrida;
16. conclusão.

38. Questões para discussão

Questão 1

Por que o Redis pode apresentar vantagem em consultas nas quais a aplicação conhece diretamente a chave do registro?

Questão 2

Qual é a diferença entre:

```
taxi:corrida:10001
```

e:

```
SELECT * FROM corrida WHERE id = 10001;
```

em termos de modelo de dados?

Questão 3

Por que o resultado do benchmark não permite afirmar que Redis é sempre mais rápido que MySQL?

Questão 4

Qual é a importância do pipeline durante a carga dos dados?

Questão 5
Por que o Redis pode ser interessante para implementar um cache?

Questão 6
Quais problemas surgem quando tentamos representar consultas relacionais complexas utilizando exclusivamente um banco chave-valor?

Questão 7
Em uma aplicação real, por que Redis e MySQL poderiam ser utilizados simultaneamente?

40. Contribuições da Atividade

Ao final do exercício, avalie os seguintes pontos:

"Qual banco é mais adequado ao padrão de acesso e às necessidades da aplicação?"

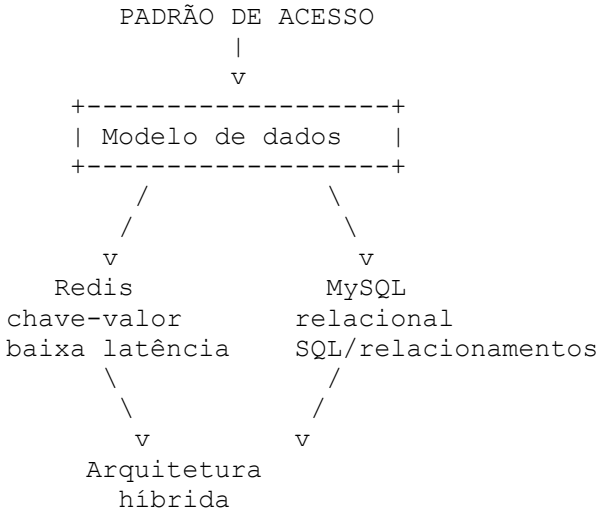
O Redis pode ser extremamente interessante quando a aplicação precisa realizar acessos rápidos por chave, manter caches, contadores, rankings ou outras estruturas especializadas.

Por outro lado, o MySQL continua sendo mais adequado para diversos cenários que exigem:

- relacionamentos;
- integridade referencial;
- consultas SQL complexas;
- agregações;
- operações envolvendo múltiplas tabelas;
- armazenamento relacional persistente.

Portanto, o experimento deverá levar os estudantes à conclusão de que **Redis e MySQL não precisam necessariamente competir**. Em uma arquitetura real, eles podem desempenhar papéis diferentes e complementares.

Conceito central da atividade:



Referências técnicas

- Redis — documentação de comandos e estruturas de dados.

- Redis — Hashes.
- Redis — cliente Python `redis-py`.
- Redis — Pipelines e transações.
- Governo Federal — repositório oficial do TaxiGov.
- MySQL — Connector/Python Developer Guide.
- MySQL — `executemany()`.